

Algoritmická složitost

- Časová, paměťová
- Umožňuje srovnávat algoritmy (když mají oba dostatečně velká vstupní data)
- „Kolikrát se prodlouží běh algoritmu po určitém zvětšení vstupu“
 - o Zvětším vstup 2krát – kolikrát se zpomalí algoritmus?
 - o Zvětším vstup o 10 – o kolik se zvětší nárok na paměť?

Na střední školu stačí mít základní pojetí a intuici, k tomu, který algoritmus bude lepší, a mít základní nástroje na důkaz „proč“.

Velikost vstupu se většinou označuje proměnnou „n“.

Značí se množinou „O“ v závislosti na funkci.

Např. $O(1)$, $O(n)$, $O(\log_2(n))$, $O(n^2)$.

Čím pomaleji roste funkce, tím lepší by měl algoritmus být.

V základu:

- „Kolikrát se zpomalí algoritmus, když se zvětší vstup o 1.“ (nebo kolikrát se zvedne nárok na paměť)
- Samotné instrukce jsou většinou zanedbatelné, např.:
 - o `num = 0`
 - o `num += 5`
 - o `if num > 2:`
- Opakované instrukce už většinou vedou ke zpomalení/větší paměti, např.:
 - o `for i in range(num): num -= 1`
 - o `while num > 0: num -= 1`
- Není ale pravidlem, že každý cyklus posune složitost „n-krát“:
 - o `for i in range(num * num * num) -> „n3-krát“`
 - o `for i in range(5) -> „5 krát“ -> konstanta`

Násobky číslem (konstanty) se u funkcí neuvádí a zpravidla stačí zapsat jen „nejsilnější“ člen (vysvětlila by hlubší teorie – asymptotika):

- $O(12) \rightarrow O(1)$
- $O(25n + 13) \rightarrow O(n)$
- $O(n^3 + n^2 + n + 1) \rightarrow O(n^3)$
- $O(n \log(n))$ zůstane stejně, nemá číselné násobky

Na procvičení – určit časovou a paměťovou složitost:

```
1  def f1(n: int) -> int:
2      return n * 5
3
4  def f2(n: int) -> int:
5      list = [n * (i+1) for i in range(5)]
6      return list[-1]
7
8  def f3(n: int) -> int:
9      list = []
10     for i in range(n * n):
11         for j in range(n):
12             for k in range(5):
13                 list.append(k + 1)
14
15     return list[-1]

18  def f4(n: int) -> int:
19     num = 0
20     for i in range(n):
21         for j in range(n):
22             for k in range(5):
23                 num = k + 1
24     return num
```

Analýzu může ovlivnit druh vstupních dat, např.:

- Řazení (sorting) se může (a pravděpodobně bude) chovat jinak s náhodným seznamem a již seřazeným seznamem
 - o Náhodné – pomalé – např. $O(n^2)$
 - o Již seřazené – stačí zkontrolovat, že je již seřazené – např. $O(n)$
- U algoritmů, které opakovaně půlí svůj vstup, jsou jasně definované hranice, kde se zpomalí, protože musí půlit vícekrát, např.:
 - o Vstupy o velikosti menší než 1024 -> $O(n)$
 - o Vstupy o velikosti 1024-8096 -> $O(n^2)$
 - o Atd.

Častým zádrhelem jsou cykly typu „hledání duplicitních prvků“:

```
# Najdi, zda se nějaký prvek v seznamu opakuje
def duplicitni(seznam: list[int]) -> bool:
    for i in range(len(seznam)):
        for j in range(i+1, len(seznam)):
            if seznam[i] == seznam[j]:
                return True

    return False
```

Jaká je časová složitost tohoto algoritmu?

Pro seznam velikosti 5 se bude funkce vykonávat následovně:

Počet opakování řádky s podmínkou, modré se vykonají, bílé ne:

	j = 0	j = 1	j = 2	j = 3	j = 4
i = 0					
i = 1					
i = 2					
i = 3					
i = 4					

Výsledek je podobný výpočtu průměrné rychlosti při zrychlování... Neboli:

$$N * N / 2$$

Což ale obsahuje konstantu, tudíž výsledek je i tak $O(n^2)$